Compiled by ab

AMBITION GURU

Computer Concept and Programming

I semester, BSc. CSIT

Syllabus

piled by ab

AMBITION GURU

Unit	Contents	Hours
1.	Computer Fundamental and Programming Methodology	3
2.	Overview of C Language	4
3.	Control Structures	6
4.	Arrays and Strings	6
5.	Functions	8
6.	Pointers	5
7.	Structures and Unions	5
8.	**File Handling in C**	**5**
9.	Introduction to Graphics	3

Practical Works

Credit hours : 3

Compiled by ab

Unit 8

(5 hrs.)

File handling in C

- Introduction, Basic Terminology Associated with Files, Types of Files, Streams and Files, Binary vs Text File ad File Buffering, File System Structures, Various Types of File Access Methods, Input and Output Operations on Files and Standard Devices, File Operations, Error Handling in Files, Command Line Arguments

Compiled by ab

Unit 8: Part 1

File Handling, File Access Method, Type of files

compiled by ab

File Handling

- File handling in C refers to reading data from or writing data to files stored on disk.
- It allows programs to store outputs permanently and retrieve inputs when needed.
- File handling makes it possible to perform operations like create, update, read, and delete files.
- In C, files are managed through the use of FILE pointers and associated functions from the <stdio.h> library.

Compiled by ab

File Handling in C

Why file handling is needed in c program?

When problem involves large volume of data and in such situations the console oriented i/o operations pose two major problems.

- It becomes cumbersome and time consuming to handle large volume of data through terminals.
- The entire data is lost when either a program is terminated or the computer is turned off.

Why use file handling?

- To store large amounts of data permanently.
- To share data between programs.
- To keep records like student data, employee information, etc.

File Access Methods

Compiled by ab

Sequential Access:

- Data is read or written one after another.
- Start from the beginning and move through the file in order.
- Example: Reading a text file line by line.

Direct (Random) Access:

- Directly jump to any location in the file without reading the previous data.
- Functions like fseek(), ftell(), and rewind() are used.
- Example: Reading the 5th record in a file without reading the first 4 records.

Difference:

- Sequential is slow if you want a random record.
- Direct is faster but more complex to manage.

Introduction: Type of Files

Compiled by ab

AMBITION GURU

- Text files
  - Contain ASCII code only
    - C-programs
- Binary files
  - Contain non-ASCII characters
    - Image, audio, video, executable, etc.

I. **Text files:** Text file is a human readable sequence of characters and the words they form that can be encoded into computer-readable format such as ASCII. A text file is also known as ASCII file and can be read by any word processor. Text files stores information in consecutive characters. These characters can be interpreted as individual data items or as a component of strings or numbers.

II. **Binary files:** A binary files is made up of machine readable symbols that is made up of 0's and 1's.The binary file must be interpreted by the program that understand in advance exactly how it is formatted. Binary files are organized into blocks containing contiguous bytes of information. These blocks represents more complex data structures, such as array and structures.

Differences between text file and binary file.

Compiled by ab

AMBITION GURU

Text File	Binary File
Text file is human readable because everything is stored in terms of text.	In binary file everything is written in terms of 0 and 1, therefore binary file is not human readable.
Every text file is terminated with EOF(End of File),whose ASCII value is 26.	There is no such special character present in the binary mode files to mark the end of file.
A newline (\n) character is converted into the carriage return-linefeed combination before being written to the disk.	In binary file, these conversions will not take place.
In text file, the text and characters are stored one character per byte. For example, the integer value 12345 will occupy 2 bytes in memory but it will occupy 5 bytes in text file.	In binary file, the integer value 12345 will occupy 2 bytes in memory as well as in file.
The extension for text file is .txt.	The extension for binary file is .dat.
File opening modes for text file are r, w, a, r+, w+ and a+.	File opening modes for binary file are rb,wb, ab, rb+,wb+ and ab+.

Compiled by ab

Unit 8: Part 2

Example, Basic file operations in C, Functions of file handling

AMBITION GURU

Create a structure called student having name, age and rollno. Input a record store it in a file called student.txt and display the information.

```
#include <stdio.h>
#include <stdlib.h>
#include<conio.h>

struct Student {
    char name[50];
    int age;
    int rollno;
};


int main() {
    struct Student s;

    FILE *fp;
    fp = fopen("student.txt", "w+");
    if (fp == NULL)
    {
        printf("File couldn't be opened");
        exit(0);
    }
```

Compiled in ab

Compiled by ab

Create a structure called student having name, age and rollno. Input a record store it in a file called student.txt and display the information.

```
printf("Enter student name: ");
gets(s.name);
printf("Enter student age: ");
scanf("%d", &s.age);
printf("Enter student roll number: ");
scanf("%d", &s.rollno);

fwrite(&s, sizeof(s), 1, fp);

fclose(fp);
fp = fopen("student.txt", "r+");
if (fp == NULL) {
     printf("File couldn't be opened for reading\n");
        exit(1);
    }
```

Compiled by ab

Create a structure called student having name, age and rollno. Input a record store it in a file called student.txt and display the information.

```
printf(“ The information of student is \n”);

fread(&s, sizeof(s), 1, fp);

printf("Name   : %s\n", s.name);
printf("Age    : %d\n", s.age);
printf("Roll No: %d\n", s.rollno);

fclose(fp);
getch();
return 0;
}
```

Compiled by ab

Basic file operations in C

compiled by ab

Basic file operations in C

The basic file operations in C are:

1. Naming a file
2. Opening a file
3. Reading data from file
4. Writing data to file and
5. Closing the file

compiled by ab

Basic file operations in C

Opening and closing a file

- **Opening a File:** Before a program can read from or write to a file, it must be opened to establish a connection between the program and the operating system, specifying the file's name and access mode (e.g., read, write).
- **File Pointer and Structure:** A file is managed using a FILE structure (defined in stdio.h), which stores details like the file's name, status, buffer size, and current position, with a file pointer referencing this structure to identify the file.

Basic file operations in C

Compiled by ab

The function **fopen()** is used to open the file. The buffer area is established by:

FILE *ptr_variable;

And file is opened by using the following syntax:

ptr_variable = fopen("file_name", " file_opening_mode");

Here, file_name is the name of the file we intend to open and opening mode specifies the purpose of

opening file.

For example:

```
FILE *fptr1,*fptr2;
fptr1=fopen("myfile.txt","w");
fptr2=fopen("testfile.dat","rb");
```

compiled by ab

Basic file operations in C

The file that was opened using **fopen()** function must be closed when no more operations are to be performed on it. After closing the file the connection between file and the program is broken. On closing the file, all buffers associated with it are flushed and can be available for other files.

Its syntax is:

```
fclose(ptr_variable);
```

for example:

fclose(fptr1);

fclose(fptr2);

compiled by ab

Basic file operations in C

Question: What is the significance of file pointer in file handling?

File Pointer

A file pointer is a pointer to a structure, which contains information about the file, including its name, current position of the file, whether the file is being read or written, and whether errors or end of the file have occurred. The user does not need to know the details, because the definitions obtained from *stdio.h* include a structure declaration called FILE. The only declaration needed for a file pointer is symbolized by

```
FILE *fptr;
```

This says that fptr is the file pointer that points to a FILE structure

Functions of file handling

Compiled by ab

fopen(), fclose(), fflush(), freopen()

fflush()

Purpose: Flushes (clears) the output buffer. Useful for forcing data to be written to a file or standard output immediately.

Syntax:

```
fflush(fp);
```

It is mostly used in output operations, not commonly with input files.

Compiled by ab

Functions of file handling

freopen()

Purpose: Closes an existing file and reopens it with a new mode or file name.

Syntax:

```
FILE *fp;
fp = freopen("newfile.txt", "w");
```

Compiled by ab

Unit 8: Part 3

File opening modes, Character input output , String input output

Compiled by ab

File opening modes

File opening modes

Compiled by ab

File opening modes specifies the way in which a file should be opened.

In other words it specifies the purpose of opening a file. Let us discuss the file opening mode for text file.

File mode	Meaning of mode	During Inexistence of file
"r"	Opens an existing file for reading purpose only	If the file does not exist, fopen() returns NULL.
"w"	Opens a file for writing purpose	If the file exists, its contents are overwritten(contents are deleted first and written). If the file does not exist, it will be created.
"a"	Opens an existing file for appending purpose (i.e. addition of new content at the end of the existing file)	If the file does not exists, it will be created.
"r+"	Opens an existing file for both reading and writing purpose.	If the file does not exist, fopen() returns NULL.
"w+"	Opens file for reading and writing purpose.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
"a+"	Opens an existing file for both reading and appending purpose.	If the file does not exists, it will be created.

File opening modes

Compiled by ab

Note: The file opening modes in binary files are similar to text mode .Character b is added to each mode to indicate the binary mode. eg. rb, wb,ab,rb+,wb+,ab+.

E.g.

```
FILE *fptr;
fptr=fopen("sample.txt", r);
```

Here, file named sample.txt is opening for reading mode only.

Similarly,

```
FILE *fptr;
fptr=fopen("hello.dat",wb+);
```

Here, file named hello.dat is opening for both reading and writing.

Compiled by ab

Reading and writing data to a file

Compiled by ab

1. Character I/O functions

Compiled by ab

Character I/O functions

Using character I/O functions data can be read from a file or written to a file one character at a time.

fgetc(): It is used to read a character from a file.

Syntax: **char_variable=fgetc(file_ptr_variable);**

fputc(): It is used to write a character to file.

Syntax: **fputc(char_variable,file_ptr_variable);**

Character I/O functions

WAP to read a characters from keyboard and write character to a file.

OR

WAP to create a file named sample.txt and write some characters to a file until user hits the enter key.

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
int main()
{
char ch;
FILE *fptr;
fptr=fopen("sample.txt","w");
if(fptr==NULL)
{
  printf("File cannot be openend");
  exit(1);
}
printf("Enter some characters");
while((ch=getchar())!='\n')
{
   fputc(ch,fptr);
}
fclose(fptr);
getch();
return 0;
}
```

Compiled by

AMBITION GURU

Character I/O functions

WAP to open the file created in above example and read character stored in it and display it to the screen.

OR

WAP to read a character from the file name sample.txt and display it on screen.

Compiled by ab

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>

int main()
{
char ch;
FILE *fptr;
fptr=fopen("sample.txt","r");
printf("The character from file are\n");
if(fptr==NULL)
{
  printf("File cannot be openend");
  exit(1);
}

while((ch=fgetc(fptr))!=EOF)
{
  putchar(ch);
}
fclose(fptr);
getch();
return 0;
}
```

AMBITION GURU

Compiled by ab

2. String Input/ Output functions

String Input/ Output functions

Compiled by ab

AMBITION GURU

Using string I/O functions, data can be read from a file or written to a file in the form of array of

characters.

fgets(): It is used to read a string from a file

syntax: fgets(string_variable,int_value,file_ptr_variable);

Here, int_value denotes the number of characters in string .The functions read a string from a file representing file_ptr_variable and stores in a variable string variable.

fputs():It is used to write a string to a file

syntax: fputs(string_variable,file_ptr_variable);

Here, contents in the string variable is written to a file representing a file_ptr_variable.

String Input/ Output functions

Write a program to create a file named “info.txt” and write “Welcome to Ambition” and display it.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fptr;
    char buffer[100]; // Buffer to hold the line

    fptr = fopen("info.txt", "w+");
    if (fptr == NULL) {
        printf("File cannot be opened.\n");
        exit(1);
    }
    fputs("Welcome to Ambition", fptr);
    rewind(fptr);
    printf("Contents of the file:\n");

    if (fgets(buffer, sizeof(buffer), fptr) != NULL) {
        printf("%s", buffer);  // Display the string
    }

    fclose(fptr);
    return 0;
}
```

Compiled by ab

AMBITION GURU

Compiled by ab

Unit 8: Part 4

Formatted input output, Record I/O

Compiled by ab

3. Formatted Input /Output

compiled by ab

Formatted Input /Output

These functions are used to read numbers, characters or string from a file or write them to a file in a format as per requirement.

fprintf(): It is formatted output function which is used to write integer, float ,char or string data to a file.

Syntax: fprintf(file_ptr_variable,”control_string”,list_variables);

fscanf(): It is formatted input function which is used to read integer ,float, char or string data from a file.

Syntax: fscanf(file_ptr_variable,”format _specifier”,&list_variables);

Compiled by ab

AMBITION GURU

Formatted Input /Output

WAP to input age, gender, marks of a student in a file called “student.txt”. Now read these information from the file display them.

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
int main()
{
char gender;
int age;
float marks;
FILE *fptr;
fptr=fopen("student.txt","w+");
if(fptr==NULL)
{
printf("File cannot be opened");
exit(1);
}
```

Compiled by ab

Formatted Input /Output

WAP to input age, gender, marks of a student in a file called “student.txt”. Now read these information from the file display them.

```
printf("Enter the gender\n");
scanf("%c",&gender);
printf("Enter the age\n");
scanf("%d",&age);
printf("Enter the marks\n") ;
scanf("%f",&marks);
fprintf(fptr,"%c\t%d\t%f",gender,age,marks);
rewind(fptr);
fscanf(fptr,"%c %d %f",&gender,&age,&marks);
printf("Gender=%c\tAge=%d\tMarks=%f",gender,age,marks);
fclose(fptr);
getch();
return 0;
}
```

Compiled by ab

4. Record I/O

AMBITION GURU

4. Record I/O

Compiled by ab

Record i/o writes numbers to files in binary format.so that integers are stored in 2 bytes ,floating point numbers are stored in 4 bytes and so on.

- It permits reading or writing multiple data values in the form of arrays, structures and array of structures once.
- Thus the arrays, structures, array of structures etc. can be read from a file or written to a file as a single unit using record i/o.

There are functions fwrite and fread for writing and reading structure (record) in a file on a disk.

compiled by ab

4. Record I/O

fwrite () used for writing entire block to a given file.

Syntax: fwrite(&ptr,size_of_array_or_structure,number_of_structure_or_array,fptr);

fread() is used to read an entire block from a given file.

Syntax: fread(&ptr, size_of_array_or_structure, number_of_structure_or_array, fptr);

where,

- ptr is the address of an array or structure to be written
- Size_of_array_or_structure is an integer value that shows the size of structure or size of array which is being read or written.
- number_of_structure_or_array is an integer value that indicates number of arrays or structures to be written to file or read from file.
- fptr is a file pointer of a file opened in binary mode.

Compiled by ab

Write a program to create structure for the following data for student (rollno, name, phone, address and semester).Read the 10 students by user and write only those students whose semester is 1 in file “student.txt”.

AMBITION GURU

Compiled by ab

Write a program to create structure for the following data for student (rollno, name, phone, address and semester).Read the 10 students by user and write only those students whose semester is 1 in file “student.txt”.

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
struct student
{
char name[20];
int rollno;
long int phone;
char address[20];
int semester;
};

int main()
{
int i;
struct student st[10];
FILE *fptr;
fptr=fopen("student.txt","w+");
if(fptr==NULL)
{
printf("File cannot be opened");
exit(1);
}
```

Write a program to create structure for the following data for student (rollno, Name, phone, address and semester).Read the 10 students by user and write only those students whose semester is 1 in file “student.txt”.

```
printf("Enter records of 10 students\n");
for(i=0;i<10;i++)
{
printf("Enter the name\n");
gets(st[i].name);
printf("Enter rollno\n");
scanf("%d",&st[i].rollno);
printf("Enter Phone number\n");
scanf("%ld",&st[i].phone);
printf("Enter the Address\n");
gets(st[i].address);
printf("Enter semester\n");
scanf("%d",&st[i].semester);
if(st[i].semester==1)
{
        fwrite(&st[i],sizeof(st[i]),1,fptr);
}
}
printf("Data written successfully");
fclose(fptr);
getch();
return 0;
}
```

Compiled by Ab

AMBITION GURU

Compiled by ab

Unit 8: Part 5

Error handling in files

Compiled by ab

Error handling in files

Error handling in files

- When working with files, it's crucial to check whether file operations like fopen(), fread(), fwrite(), etc. succeed. If not handled properly, they can crash your program.

Compiled by ab

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;

    fp = fopen("data.txt", "r");

    if (fp == NULL) {
        printf("Error opening file");
        exit(1);  // Exit with failure
    }

    // File operations go here...
    fclose(fp);
    return 0;
}
```

Note:

- fopen() returns NULL on failure.
- printf("message") prints the system error message.
- exit(1) exits the program if the file can't be opened.

Questions

Compiled by ab

1. Define file pointer. Write down the syntax for fopen(), fclose(),fprint() and fscanf().
2. WAP in C to write the following content in the file named "goal.txt“

   " I am learning C programming“
3. WAP to create a structure named employee with members as emp_id, emp_name and emp_salary. Take an input of n employees and write it to the file named employee.txt. Also, read and display the information of each employee.
4. Explain different file opening modes.
5. Describe the different types of I/O functions used in file handling with syntax.
6. Write a program that simply reads data from a file.
7. Write a program that simply write data to a file.
8. Why do we need data files? What are the different file opening modes? Write a program that reads data from a file “input.txt” and writes to “output.txt” file.

Compiled by ab

AMBITION GURU

THANK YOU

Any Queries ?